

ASSESSMENT - SET 4

Data Scientist / Machine Learning Engineer Evaluation

PART A: Multiple Choice Questions

Q1. In a RAG pipeline, "late interaction" models such as ColBERT differ from bi-encoder models in that:

- A) ColBERT uses a cross-encoder that jointly attends to query and document tokens at retrieval time
- B) ColBERT produces per-token embeddings for both query and document, and scores relevance using a MaxSim operation over all query-document token pairs, enabling richer matching without full cross-encoder cost
- C) ColBERT indexes documents as a single pooled vector, identical to a standard bi-encoder
- D) ColBERT only works on short passages of fewer than 128 tokens

Q2. Document summarization as a pre-processing step in RAG ("summary indexing") is useful when:

- A) The documents are already short enough to fit in the LLM's context window
- B) Source documents are very long and dense; summaries capture the key ideas compactly, improving retrieval precision compared to chunking a large document into many small pieces
- C) The embedding model's vocabulary does not support the document's language
- D) BM25 retrieval is used instead of vector search

Q3. In an LLM-based multi-agent system, the "orchestrator" agent's primary responsibility is to:

- A) Execute low-level tool calls such as web search or code execution
- B) Decompose the overall task, assign sub-tasks to specialised worker agents, and synthesise their outputs into a final response
- C) Generate the initial user query and forward it to the vector store
- D) Fine-tune sub-agents on task-specific data at runtime

Q4. XLNet differs from BERT in its pre-training approach by:

- A) Using a standard left-to-right causal language model objective like GPT
- B) Using a permutation language modeling objective that predicts tokens in a random factorisation order while maintaining full bidirectional context through attention masking, avoiding BERT's [MASK] token discrepancy

- C) Training on 10x more data without changing the objective
- D) Removing positional embeddings and relying solely on relative attention biases

Q5. The primary limitation of using a decoder-only model for extractive QA compared to an encoder-only model is:

- A) Decoder-only models cannot process passages longer than encoder-only models
- B) Causal attention means each token can only attend to prior tokens, so a token in the middle of a passage cannot incorporate information from tokens that appear after it - limiting contextual understanding of the passage
- C) Decoder-only models do not have a softmax layer and cannot output span probabilities
- D) Encoder-only models have a special CLS token that decoder-only models lack, making span extraction impossible

Q6. In the original transformer (Vaswani et al., 2017), positional encoding uses sinusoids of different frequencies because:

- A) Sinusoidal encodings can be learned during training, unlike fixed encodings
- B) Each dimension encodes position at a different frequency, allowing the model to attend to relative positions by linear combinations of the encoding vectors, and the pattern generalises to lengths not seen during training
- C) Sinusoidal functions avoid vanishing gradients during backpropagation through the positional signal
- D) The use of sine and cosine doubles the information capacity of the positional representation

Q7. Multi-Query Attention (MQA) reduces memory and inference latency compared to MHA by:

- A) Reducing the number of query heads while keeping key-value heads unchanged
- B) Using a single shared key and a single shared value head across all query heads, dramatically reducing KV-cache size and the memory bandwidth needed to load keys and values during decoding
- C) Caching only the most recent 1024 positions in the KV-cache regardless of sequence length
- D) Applying linear attention to replace the softmax operation in all heads

Q8. Curriculum learning in the context of LLM pre-training refers to:

- A) Training on easy domain data first and progressively introducing harder, noisier, or more complex data to stabilise and improve training
- B) Interleaving supervised fine-tuning with pre-training to improve sample efficiency
- C) Using a teacher model to generate soft labels for all pre-training data before training begins
- D) Scheduling the learning rate to follow a cosine decay with warm restarts

Q9. In LLM inference, "key-value cache eviction" strategies (e.g., H2O, SnapKV) are used to:

- A) Delete key-value entries from the cache when GPU temperature exceeds a threshold
- B) Selectively discard KV entries for tokens deemed less important (e.g., based on accumulated attention scores) to keep the KV-cache within a fixed budget, enabling very long-context inference without unbounded memory growth
- C) Evict the KV-cache to CPU RAM when GPU memory is full and reload it as needed
- D) Compress KV-cache entries from FP16 to INT8 on-the-fly to save memory

Q10. The Adam optimiser's bias-correction step (dividing by $1 - \beta^t$) is necessary because:

- A) Without it, the learning rate would explode in the first few iterations
- B) At initialisation, m and v are set to zero; the exponential moving average is biased towards zero in early steps, and the correction inflates the estimates to their true expected values, preventing artificially small weight updates
- C) The bias correction converts the gradient estimates from second-order to first-order
- D) It prevents the adaptive learning rate from decreasing below a minimum threshold

Q11. Continued pre-training (domain-adaptive pre-training) differs from instruction fine-tuning in that:

- A) Continued pre-training uses labelled (input, output) pairs; instruction fine-tuning uses raw text
- B) Continued pre-training trains the model on large amounts of raw domain text using the original pre-training objective to inject domain knowledge; instruction fine-tuning uses curated (instruction, response) pairs to improve instruction-following behaviour
- C) Only instruction fine-tuning updates the transformer weights; continued pre-training only updates the embedding layer
- D) Continued pre-training is done with a frozen base model using adapters only

Q12. ORPO (Odds Ratio Preference Optimization) combines supervised fine-tuning and preference alignment by:

- A) Running SFT and DPO in two separate sequential training stages
- B) Incorporating a penalty term based on the log odds ratio of chosen vs rejected responses directly into the SFT cross-entropy loss, so alignment and instruction fine-tuning happen in a single training pass without a reference model
- C) Using odds ratios from A/B testing of deployed model outputs to create preference data
- D) Fine-tuning the model with an odds-ratio reward model trained on human preference labels

Q13. Weight clustering (k-means quantization) differs from uniform quantization in that:

- A) Weight clustering rounds every weight to the nearest power of two
- B) Weight clustering groups weights into k clusters using k-means, representing all weights in a cluster with the centroid - allowing non-uniformly spaced quantization levels that better fit the actual weight distribution
- C) Uniform quantization uses a different codebook per layer while clustering uses one global codebook
- D) Weight clustering is only applicable to attention weight matrices, not FFN layers

Q14. The main benefit of FP8 training (as used in H100 GPU Tensor Cores) over INT8 training is:

- A) FP8 uses integer arithmetic which is faster than floating-point operations
- B) FP8 retains a floating-point representation with an exponent field, handling the dynamic range of both forward activations and backward gradients without requiring separate integer scale factors
- C) FP8 reduces memory by 8x compared to FP16
- D) FP8 is lossless - it represents all FP32 values without any approximation

Q15. A translation model is evaluated with BLEU on a test set and scores 42. Without additional context, this number:

- A) Means the model translates correctly 42% of the time
- B) Is not directly interpretable in isolation - BLEU scores are only meaningful relative to other systems or human translators on the same test set with the same tokenization and reference configuration
- C) Indicates the model is below human-level since human translators score 100
- D) Means 42% of the model's output n-grams exactly match the references

Q16. The WMT shared task evaluation campaigns are significant in MT research because:

- A) They provide standardised test sets, baselines, reference translations, and human evaluation scores that allow fair comparison of MT systems across research groups
- B) They define the BLEU metric and update its formula every year
- C) They exclusively evaluate low-resource language pairs not covered by academic benchmarks
- D) They replace human evaluation with automatic metrics to reduce cost

Q17. Reference-free evaluation metrics for machine translation (e.g., COMET-QE, CometKiwi) are advantageous in production settings because:

- A) They always outperform reference-based metrics in correlation with human judgments
- B) They estimate translation quality using only the source and hypothesis (no reference)

required), enabling continuous monitoring of MT quality in production where human references are unavailable

- C) They do not require any pre-trained model and can be computed with simple rule-based heuristics
- D) They are language-agnostic and work equally well for all language pairs without retraining

Q18. Layer normalization is applied differently in Pre-LN (GPT-2 style) vs Post-LN (original transformer). Pre-LN is preferred in large models because:

- A) Post-LN normalizes between the attention and FFN sub-layers while Pre-LN normalizes only at the output
- B) Pre-LN normalizes the input to each sub-layer before the residual is applied, producing more stable gradient flow during training and allowing training with higher learning rates; Post-LN can cause gradient explosion in very deep networks
- C) Pre-LN removes the need for residual connections since normalization already prevents vanishing gradients
- D) Post-LN is only applicable to encoder-only models and cannot be used in decoder-only architectures

Q19. In the context of quantized LLM deployment, "dequantization overhead" at inference time refers to:

- A) The time needed to convert the output logits from INT8 to FP32 before the softmax
- B) The compute cost of converting quantized weights back to FP16/BF16 on-the-fly before each matrix multiplication, since most GPU GEMM kernels operate in FP16; this conversion can offset quantization speedups if not implemented with fused kernels
- C) The latency of downloading quantized model weights from a remote model hub
- D) The cost of running a calibration pass to compute per-layer quantization scale factors

Q20. In RAG, the "lost in the middle" phenomenon refers to:

- A) Chunks that are retrieved but never appear in the final context window due to reranker filtering
- B) The empirically observed tendency of LLMs to under-utilise information placed in the middle of a long context window, paying more attention to content at the beginning and end - which has implications for how retrieved chunks should be ordered when constructing the prompt
- C) Queries that partially match multiple chunks but fully match none, resulting in poor retrieval
- D) The decoder ignoring retrieved context and relying entirely on parametric knowledge

PART B: Coding Questions

Coding Q1 - Count of Smaller Numbers After Self

Problem Statement

Given an integer array `nums`, for each element `nums[i]`, count the number of elements to its right that are strictly smaller than `nums[i]`.

Return an integer array `counts` where `counts[i]` is the count for `nums[i]`.

You must solve this using binary search on a dynamically maintained sorted list - do not use a brute-force $O(n^2)$ nested loop as your primary solution.

Constraints

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- Expected time complexity: $O(n \log n)$

Open Test Cases (Visible to Candidate)

```
Input:  nums = [5, 2, 6, 1]
Output: [2, 1, 1, 0]
```

```
Input:  nums = [1]
Output: [0]
```

Coding Q2 - Longest Repeating Character Replacement

Problem Statement

You are given a string `s` consisting of uppercase English letters and an integer `k`.

You may choose at most `k` characters in the string and replace them with any uppercase letter.

After performing at most `k` replacements, find the length of the longest substring that contains only one distinct character.

Constraints

- $1 \leq \text{s.length} \leq 10^5$
- `s` consists of uppercase English letters only
- $0 \leq k \leq \text{s.length}$
- Expected time complexity: $O(n)$

Open Test Cases (Visible to Candidate)

```
Input:  s = "ABAB",  k = 2
Output: 4
```

```
Input:  s = "AABABBA",  k = 1
Output: 4
```

Coding Q3 - Subarray Sum Equals K

Problem Statement

Given an integer array `nums` and an integer `k`, return the total number of contiguous subarrays whose elements sum to exactly `k`.

The array may contain negative numbers.

You must solve this in $O(n)$ time - do not use a brute-force $O(n^2)$ approach as your primary solution.

Constraints

- $1 \leq \text{nums.length} \leq 2 \times 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$
- $-10^7 \leq k \leq 10^7$
- Expected time complexity: $O(n)$

Open Test Cases (Visible to Candidate)

```
Input:  nums = [1, 1, 1],  k = 2
Output: 2
```

```
Input:  nums = [1, 2, 3],  k = 3
Output: 2
```